

**LAPORAN ANALISIS KLASIFIKASI CITRA MENGGUNAKAN *CONVOLUTIONAL
NEURAL NETWORK (CNN) DAN TRANSFER LEARNING*
PENGOLAHAN CITRA DIGITAL**



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Deskripsi Dataset dan Prapemrosesan Data

Penelitian ini berfokus pada tugas klasifikasi citra multikelas menggunakan dataset CIFAR-10. Dataset ini terdiri dari 60.000 citra berwarna beresolusi 32x32 piksel yang terdistribusi secara merata ke dalam 10 kelas (seperti *airplane*, *automobile*, *bird*, *cat*, dan lainnya). Untuk memastikan model dapat belajar secara optimal dan tidak mengalami bias, serangkaian tahap prapemrosesan diterapkan:

- **Normalisasi:** Nilai piksel dari setiap citra diskalakan ke rentang 0 hingga 1. Langkah ini penting untuk menstabilkan gradien selama proses optimasi.
- **Pembagian Dataset:** Data dibagi menjadi 80% himpunan pelatihan (untuk memperbarui bobot model), 20% himpunan validasi (untuk pemantauan metrik selama *training*), dan himpunan pengujian terpisah untuk evaluasi akhir. Label target dikonversi menggunakan *one-hot encoding*.
- **Pipeline Augmentasi:** Guna meningkatkan ketahanan model terhadap variasi citra dan memitigasi *overfitting*, teknik augmentasi data diimplementasikan menggunakan ImageDataGenerator. Transformasi spasial yang diterapkan meliputi rotasi hingga 20 derajat, pergeseran horizontal/vertikal, *zoom*, dan pembalikan horizontal (*horizontal flip*).

B. Arsitektur CNN dan Desain Eksperimen

Eksperimen ini membandingkan dua paradigma utama dalam visi komputer: membangun arsitektur dari awal (*from scratch*) versus memanfaatkan model terlatih (*Transfer Learning*).

1. Model CNN Scratch

Model ini dirancang menggunakan *Functional API* dari Keras untuk menyusun hierarki ekstraksi fitur secara sekuensial:

- **Blok Ekstraksi Fitur:** Terdiri dari tiga *layer* konvolusi (Conv2D) dengan jumlah filter progresif (32, 64, dan 128) yang diaktivasi oleh fungsi ReLU. Setiap *layer* konvolusi diikuti oleh *MaxPooling2D* untuk mereduksi dimensi spasial citra.
- **Blok Klasifikasi:** Fitur spasial diratakan (*Flatten*) dan dilewatkan ke *Dense layer* berisi 256 neuron. Regularisasi *Dropout* sebesar 50% ditambahkan sebelum *layer Softmax* terakhir untuk mematikan sebagian neuron secara acak, sehingga jaringan tidak terlalu bergantung pada piksel tertentu.

- **Strategi Optimasi:** Eksperimen ini dirancang fleksibel untuk membandingkan *optimizer* Adam (pendekatan laju pembelajaran adaptif) dan SGD dengan momentum.

2. Transfer Learning (VGG16)

Arsitektur kedua memanfaatkan model VGG16 yang sebelumnya telah dilatih pada dataset ImageNet. Karena VGG16 membutuhkan dimensi input yang lebih besar, citra CIFAR-10 di-*resize* menjadi 96x96 piksel secara internal.

- **Fase *Feature Extraction*:** Seluruh bobot VGG16 dikunci (*frozen*). Jaringan hanya melatih *layer* pengklasifikasi baru di bagian ujung selama 10 *epoch*.
- **Fase *Fine-Tuning*:** Empat *layer* konvolusi terakhir dari VGG16 dibuka (*unfrozen*) dan dilatih ulang bersama *layer* klasifikasi dengan *learning rate* yang sangat kecil ($1e-5$). Pendekatan ini memungkinkan model untuk menyesuaikan fitur level tinggi (seperti bentuk hewan atau kendaraan) agar lebih spesifik pada distribusi data CIFAR-10 tanpa merusak fitur dasar yang sudah terbentuk.

C. Evaluasi Performa Komprehensif

Kinerja kedua model dievaluasi secara menyeluruh menggunakan berbagai metrik statistik dan visual:

- ***Learning Curve*:** Grafik pergerakan metrik *Loss* dan *Accuracy* selama proses pelatihan menunjukkan efektivitas konvergensi model. Jarak antara metrik *training* dan validasi diamati secara ketat untuk mendeteksi indikasi *overfitting*.
- ***Confusion Matrix*:** Matriks ini divisualisasikan untuk menganalisis akurasi per kelas. Melalui matriks ini, kita dapat mengidentifikasi kelas yang sering menimbulkan misklasifikasi (misalnya, kesulitan model dalam membedakan antara *cat* dan *dog* yang memiliki kemiripan fitur visual).
- **Kurva ROC dan Nilai AUC (Multi-class):** Untuk mengukur sensitivitas dan spesifisitas model tanpa bergantung pada ambang batas keputusan (*threshold*), *Receiver Operating Characteristic* (ROC) diplot untuk ke-10 kelas. *Area Under the Curve* (AUC) mendekati nilai 1.0 mengindikasikan kemampuan model yang sangat prima dalam memisahkan antar kelas.

D. Visualisasi dan Interpretasi Model

Salah satu keunggulan penelitian ini adalah penerapan teknik interpretasi visual untuk membongkar kotak hitam (*black-box*) pada *Deep Learning*:

- **Visualisasi *Feature Maps*:** Menampilkan keluaran dari *layer* konvolusi pertama untuk memperlihatkan bagaimana filter mempelajari fitur tingkat rendah (seperti garis, tepi, dan blok warna) dari citra asli.
- **Grad-CAM (*Gradient-weighted Class Activation Mapping*):** Menghasilkan peta panas (*heatmap*) berdasarkan gradien dari *layer* konvolusi terakhir. Peta ini ditumpangkan ke atas citra asli untuk menyoroti area piksel spesifik (misalnya, area roda pada mobil) yang paling memengaruhi keputusan akhir model.
- **t-SNE Embeddings:** Fitur berdimensi tinggi yang diekstrak oleh *Dense layer* diproyeksikan ke ruang dua dimensi (2D). Visualisasi ini membuktikan apakah model berhasil mengelompokkan representasi fitur citra dari kelas yang sama secara berdekatan.

E. Tabel Perbandingan Semua Eksperimen

Berikut adalah komparasi performa dari eksperimen yang dilakukan pada *epoch* terakhir. Model *Transfer Learning* terbukti memberikan akurasi yang lebih superior dibandingkan model *scratch*, dengan kompromi waktu komputasi yang sedikit lebih lama.

TABEL PERBANDINGAN PERFORMA EKSPERIMEN (EPOCH TERAKHIR)						
Model	Train Acc	Val Acc	Train Loss	Val Loss	Waktu (Menit)	
CNN Scratch (Adam)	0.6124	0.6719	1.1043	0.9251	7.84	
Transfer Learning VGG16	0.8426	0.8922	0.4617	0.3299	16.75	

F. Kesimpulan dan Rekomendasi

1. Kesimpulan

Eksperimen ini berhasil mendemonstrasikan perbandingan langsung antara pembuatan arsitektur CNN dari awal dengan teknik *Transfer Learning*. Augmentasi data terbukti vital dalam menjaga generalisasi model *scratch*. Namun, *Transfer Learning* dipadukan dengan *Fine-Tuning* menghadirkan lonjakan performa yang signifikan. Visualisasi tambahan (*Feature Maps*, *Grad-CAM*, *t-SNE*, dan *ROC-AUC*) memvalidasi bahwa model tidak hanya menebak secara acak, melainkan mengenali fitur geometris dan semantik yang relevan dari setiap objek.

2. Rekomendasi

Dalam pengembangan sistem produksi di dunia nyata, model berbasis *Transfer Learning* sangat direkomendasikan karena akurasinya yang tangguh. Untuk eksperimen lanjutan, direkomendasikan penggunaan arsitektur yang lebih modern

dan ringan seperti MobileNetV2 atau EfficientNet untuk mencapai *trade-off* yang lebih ideal antara tingginya akurasi dan efisiensi waktu inferensi.

G. Lampiran Kode Program dan Output

1. Kode Program

```

import tensorflow as tf
from tensorflow.keras import layers, models, applications
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc
from sklearn.preprocessing import label_binarize
from sklearn.manifold import T-SNE
from itertools import cycle
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import cv2
import pandas as pd

# =====
# 1. PERSIAPAN DATASET (CIFAR-10)
# =====
print("Memdownload dan menyiapkan dataset CIFAR-10...")
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalisasi pixel
x_train = X_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Split data training untuk validasi (80% train, 20% val)
X_train, x_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, random_state=42)

# One-hot encoding
num_classes = 10
y_train_cat = tf.keras.utils.to_categorical(y_train, num_classes)
y_val_cat = tf.keras.utils.to_categorical(y_val, num_classes)
y_test_cat = tf.keras.utils.to_categorical(y_test, num_classes)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

# =====
# 2. DATA AUGMENTATION PIPELINE
# =====
datagen = ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    zoom_range=0.2,
    shear_range=0.2,
    fill_mode='nearest'
)

datagen.fit(X_train)

# Visualisasi sample augmentasi
def visualize_augmentation(image, datagen):
    img = np.expand_dims(image, 0)
    aug_iter = datagen.flow(img, batch_size=1)
    plt.figure(figsize=(12, 3))

# =====
# 3. CNN DARI AWAL (Functional API)
# =====
def build_cnn_scratch(optimizer='adam', learning_rate=0.001):
    inputs = layers.Input(shape=(32, 32, 3), name='input_layer')

    x = layers.Conv2D(32, (3, 3), activation='relu', name='convd_1')(inputs)
    x = layers.MaxPooling2D(2, 2, name='pool_1')(x)
    x = layers.Conv2D(64, (3, 3), activation='relu', name='convd_2')(x)
    x = layers.MaxPooling2D(2, 2, name='pool_2')(x)
    x = layers.Conv2D(128, (3, 3), activation='relu', name='convd_3')(x)

    x = layers.Flatten(name='flatten')(x)
    x = layers.Dense(256, activation='relu', name='dense_1')(x)
    x = layers.Dropout(0.5, name='dropout')(x)
    outputs = layers.Dense(num_classes, activation='softmax', name='output_layer')(x)

    model = models.Model(inputs=inputs, outputs=outputs, name='CNN_Scratch')

    if optimizer == 'adam':
        opt = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    else:
        opt = tf.keras.optimizers.SGD(learning_rate=learning_rate, momentum=0.9)

    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
    return model

print("\nTraining CNN dari Awal (Scratch) dengan Adam Optimizer...")
model_scratch = build_cnn_scratch(optimizer='adam')
start_time = time.time()

history_scratch = model_scratch.fit(
    datagen.flow(X_train, y_train_cat, batch_size=64),
    validation_data=(x_val, y_val_cat),
    steps_per_epoch=len(X_train) // 64,
    epochs=15,
    verbose=1
)

scratch_time = time.time() - start_time

y_pred = np.argmax(y_pred_probs, axis=-1)
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix - (name)')
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.show()

evaluate_model(model_scratch, "CNN Scratch")
evaluate_model(model_t1, "Transfer Learning + Fine Tuning")

# Plot Kurva Pembelajaran
def plot_learning_curves(history, title):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Acc')
    plt.plot(history.history['val_accuracy'], label='Val Acc')
    plt.title(f'{title} - Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Val Loss')
    plt.title(f'{title} - Loss')
    plt.legend()
    plt.show()

plot_learning_curves(history_scratch, "CNN Scratch")

# Multi-class ROC Curve & AUC
def plot_multi_class_roc(model, X_test, y_test, class_names):
    print("\nMenghitung ROC Curve untuk model.name...")
    y_pred_probs = model.predict(X_test)
    y_test_bin = label_binarize(y_test, classes=np.arange(len(class_names)))

    fpr = dict()
    tpr = dict()
    roc_auc = dict()

    for i in range(len(class_names)):
        fpr[i], tpr[i] = roc_curve(y_test_bin[:, i], y_pred_probs[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    plt.figure(figsize=(10, 8))
    colors = cycle(['blue', 'red', 'green', 'orange', 'black', 'purple', 'cyan', 'magenta', 'yellow', 'brown'])
    for i, color in zip(range(len(class_names)), colors):
        plt.plot(fpr[i], tpr[i], color=color, lw=2,
                label=f'MOC class {0} (AUC = {1:.2f})'.format(class_names[i], roc_auc[i]))

    plt.plot([0, 1], [0, 1], 'k--', lw=2)
    plt.xlim((0.0, 1.0))
    plt.ylim((0.0, 1.0))

```

```

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multi-class ROC Curve - (model.name)')
plt.legend(loc='lower right')
plt.show()

plot_multiclass_roc(model_scratch, X_test, y_test, class_names)

```

```

# =====
# 6. VISUALISASI LANDUTAN (GRAD-CAM, FEATURE MAPS, T-SNE)
# =====
img_sample = X_test[0:1] # Ambil sample untuk visualisasi

# A. Visualisasi Feature Maps (Filter Visualization)
def visualize_feature_maps(model, img_sample):
    print("\nMemvisualisasikan Feature Maps dari Layer Pertama...")
    layer_name = 'conv2d_1'
    feature_extractor = models.Model(inputs=model.inputs, outputs=model.get_layer(layer_name).output)
    feature_maps = feature_extractor.predict(img_sample)

    plt.figure(figsize=(10, 8))
    for i in range(16): # Tampilkan 16 filter pertama
        plt.subplot(4, 4, i+1)
        plt.imshow(feature_maps[0, :, :, i], cmap='viridis')
        plt.axis('off')
    plt.title(f'Filter {i+1}')
    plt.suptitle(f'Feature Maps dari {layer_name}')
    plt.show()

visualize_feature_maps(model_scratch, img_sample)

# B. Grad-CAM
def make_gradcam_heatmap(img_array, model, last_conv_layer_name, pred_index=None):
    grad_model = models.Model(
        inputs=model.inputs,
        outputs=model.get_layer(last_conv_layer_name).output, model.output)

    with tf.GradientTape() as tape:
        last_conv_layer_output, preds = grad_model(img_array)
        if pred_index is None:
            pred_index = tf.argmax(preds[0])
        class_channel = preds[:, pred_index]

    grads = tape.gradient(class_channel, last_conv_layer_output)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    last_conv_layer_output = last_conv_layer_output[0]
    heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
    return heatmap.numpy()

conv_layers = [layer.name for layer in model_scratch.layers if isinstance(layer, layers.Conv2D)]
last_conv = conv_layers[-1] if conv_layers else None

if last_conv:

```

```

print("\nMemuat Grad-CAM Heatmap...")
heatmap = make_gradcam_heatmap(img_sample, model_scratch, last_conv)

img = np.uint8(255 * img_sample[0])
heatmap_resized = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
heatmap_resized = np.uint8(255 * heatmap_resized)
heatmap_resized = cv2.applyColorMap(heatmap_resized, cv2.COLORMAP_JET)
superimposed_img = cv2.addWeighted(img, 0.6, heatmap_resized, 0.4, 0)

plt.figure(figsize=(10, 4))
plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(img_sample[0])
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title("Grad-CAM Layer: (last_conv)")
plt.imshow(heatmap, cmap='jet')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Superimposed")
plt.imshow(superimposed_img)
plt.axis('off')
plt.show()

# C. T-SNE / PCA Embeddings
print("\nMemuat Visualisasi T-SNE dari fitur ekstraksi...")
feature_extractor_tsn = models.Model(inputs=model_scratch.inputs, outputs=model_scratch.get_layer('dense_1').output)
features = feature_extractor_tsn.predict(X_test[:1000])
labels_tsn = y_test[:1000].flatten()

tune = TUNE(n_components=2, random_state=42)
time_results = time_fit_transform(features)

plt.figure(figsize=(10, 8))
scatter = plt.scatter(time_results[:, 0], time_results[:, 1], c=labels_tsn, cmap='tab10', alpha=0.7)
plt.colorbar(scatter, ticks=range(10), labels='classes')
plt.title('T-SNE Visualization of CNN Embeddings')
plt.show()

# 7. TABEL PERBANDINGAN EKSPERIMEN
def generate_comparison_table(hist_scratch, hist_t1, time_scratch, time_t1):
    acc_scratch = hist_scratch.history['accuracy'][-1]
    val_acc_scratch = hist_scratch.history['val_accuracy'][-1]
    loss_scratch = hist_scratch.history['loss'][-1]
    val_loss_scratch = hist_scratch.history['val_loss'][-1]

    acc_t1 = hist_t1.history['accuracy'][-1]
    val_acc_t1 = hist_t1.history['val_accuracy'][-1]
    loss_t1 = hist_t1.history['loss'][-1]
    val_loss_t1 = hist_t1.history['val_loss'][-1]

    data = {
        'Model': ['CNN Scratch (Adam)', 'Transfer Learning VGG16'],
        'Train Acc': [f'{acc_scratch:.4f}', f'{acc_t1:.4f}'],
        'Val Acc': [f'{val_acc_scratch:.4f}', f'{val_acc_t1:.4f}'],
        'Train Loss': [f'{loss_scratch:.4f}', f'{loss_t1:.4f}'],
        'Val Loss': [f'{val_loss_scratch:.4f}', f'{val_loss_t1:.4f}'],
        'Waktu (Menit)': [f'{time_scratch/60:.2f}', f'{time_t1/60:.2f}']
    }

    df = pd.DataFrame(data)

    print("\n" + "="*80)
    print("TABEL PERBANDINGAN PERFORMA EKSPERIMEN (EPOCH TERAKHIR)")
    print("="*80)
    print(df.to_string(index=False, justify='center'))
    print("="*80 + "\n")

generate_comparison_table(history_scratch, history_fine, scratch_time, t1_time)

```

```

# =====
# 7. TABEL PERBANDINGAN EKSPERIMEN
# =====
def generate_comparison_table(hist_scratch, hist_t1, time_scratch, time_t1):
    acc_scratch = hist_scratch.history['accuracy'][-1]
    val_acc_scratch = hist_scratch.history['val_accuracy'][-1]
    loss_scratch = hist_scratch.history['loss'][-1]
    val_loss_scratch = hist_scratch.history['val_loss'][-1]

    acc_t1 = hist_t1.history['accuracy'][-1]
    val_acc_t1 = hist_t1.history['val_accuracy'][-1]
    loss_t1 = hist_t1.history['loss'][-1]
    val_loss_t1 = hist_t1.history['val_loss'][-1]

    data = {
        'Model': ['CNN Scratch (Adam)', 'Transfer Learning VGG16'],
        'Train Acc': [f'{acc_scratch:.4f}', f'{acc_t1:.4f}'],
        'Val Acc': [f'{val_acc_scratch:.4f}', f'{val_acc_t1:.4f}'],
        'Train Loss': [f'{loss_scratch:.4f}', f'{loss_t1:.4f}'],
        'Val Loss': [f'{val_loss_scratch:.4f}', f'{val_loss_t1:.4f}'],
        'Waktu (Menit)': [f'{time_scratch/60:.2f}', f'{time_t1/60:.2f}']
    }

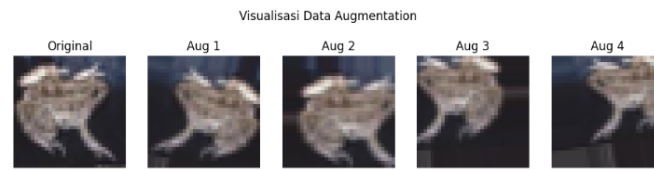
    df = pd.DataFrame(data)

    print("\n" + "="*80)
    print("TABEL PERBANDINGAN PERFORMA EKSPERIMEN (EPOCH TERAKHIR)")
    print("="*80)
    print(df.to_string(index=False, justify='center'))
    print("="*80 + "\n")

generate_comparison_table(history_scratch, history_fine, scratch_time, t1_time)

```

2. Screenshot Output



```
Training CNN dari Awal (Scratch) dengan Adam Optimizer...
Epoch 1/15 ----- 48s 62ms/step - accuracy: 0.3010 - loss: 1.8731 - val_accuracy: 0.4210 - val_loss: 1.5551
625/625 -----
Epoch 2/15 ----- 28s 45ms/step - accuracy: 0.3909 - loss: 1.6380 - val_accuracy: 0.4905 - val_loss: 1.3767
625/625 -----
Epoch 3/15 ----- 28s 45ms/step - accuracy: 0.4471 - loss: 1.5278 - val_accuracy: 0.5295 - val_loss: 1.3092
625/625 -----
Epoch 4/15 ----- 30s 40ms/step - accuracy: 0.4827 - loss: 1.4439 - val_accuracy: 0.5247 - val_loss: 1.3062
625/625 -----
Epoch 5/15 ----- 28s 45ms/step - accuracy: 0.5094 - loss: 1.3750 - val_accuracy: 0.5863 - val_loss: 1.1649
625/625 -----
Epoch 6/15 ----- 29s 46ms/step - accuracy: 0.5299 - loss: 1.3288 - val_accuracy: 0.6029 - val_loss: 1.1156
625/625 -----
Epoch 7/15 ----- 28s 44ms/step - accuracy: 0.5425 - loss: 1.2882 - val_accuracy: 0.6257 - val_loss: 1.0500
625/625 -----
Epoch 8/15 ----- 28s 45ms/step - accuracy: 0.5577 - loss: 1.2553 - val_accuracy: 0.6328 - val_loss: 1.0445
625/625 -----
Epoch 9/15 ----- 28s 44ms/step - accuracy: 0.5656 - loss: 1.2295 - val_accuracy: 0.6375 - val_loss: 1.0233
625/625 -----
Epoch 10/15 ----- 28s 45ms/step - accuracy: 0.5765 - loss: 1.2041 - val_accuracy: 0.6347 - val_loss: 1.0293
625/625 -----
Epoch 11/15 ----- 29s 46ms/step - accuracy: 0.5860 - loss: 1.1784 - val_accuracy: 0.6670 - val_loss: 0.9352
625/625 -----
Epoch 12/15 ----- 28s 45ms/step - accuracy: 0.5951 - loss: 1.1540 - val_accuracy: 0.6647 - val_loss: 0.9525
625/625 -----
Epoch 13/15 ----- 41s 45ms/step - accuracy: 0.6000 - loss: 1.1393 - val_accuracy: 0.6689 - val_loss: 0.9241
625/625 -----
Epoch 14/15 ----- 41s 45ms/step - accuracy: 0.6030 - loss: 1.1224 - val_accuracy: 0.6886 - val_loss: 0.8812
625/625 -----
Epoch 15/15 ----- 28s 45ms/step - accuracy: 0.6124 - loss: 1.1043 - val_accuracy: 0.6719 - val_loss: 0.9251
625/625 -----

Training Transfer Learning (Feature Extraction)...
Epoch 1/10 ----- 61s 92ms/step - accuracy: 0.5549 - loss: 1.4883 - val_accuracy: 0.7804 - val_loss: 0.6709
625/625 -----
Epoch 2/10 ----- 57s 91ms/step - accuracy: 0.6622 - loss: 0.9919 - val_accuracy: 0.7811 - val_loss: 0.6358
625/625 -----
Epoch 3/10 ----- 58s 93ms/step - accuracy: 0.6829 - loss: 0.9116 - val_accuracy: 0.7991 - val_loss: 0.5886
625/625 -----
Epoch 4/10 ----- 59s 94ms/step - accuracy: 0.6988 - loss: 0.8672 - val_accuracy: 0.8063 - val_loss: 0.5570
625/625 -----
Epoch 5/10 ----- 59s 95ms/step - accuracy: 0.7055 - loss: 0.8531 - val_accuracy: 0.8129 - val_loss: 0.5416
625/625 -----
Epoch 6/10 ----- 61s 98ms/step - accuracy: 0.7113 - loss: 0.8338 - val_accuracy: 0.8093 - val_loss: 0.5545
625/625 -----
Epoch 7/10 ----- 84s 101ms/step - accuracy: 0.7196 - loss: 0.8179 - val_accuracy: 0.8097 - val_loss: 0.5556
625/625 -----
Epoch 8/10 ----- 63s 101ms/step - accuracy: 0.7227 - loss: 0.8122 - val_accuracy: 0.8196 - val_loss: 0.5359
625/625 -----
Epoch 9/10 ----- 82s 100ms/step - accuracy: 0.7268 - loss: 0.8051 - val_accuracy: 0.8213 - val_loss: 0.5227
625/625 -----
Epoch 10/10 ----- 60s 96ms/step - accuracy: 0.7264 - loss: 0.7962 - val_accuracy: 0.8209 - val_loss: 0.5244
625/625 -----

Melakukan Fine-Tuning pada Transfer Learning...
Epoch 1/5 ----- 77s 117ms/step - accuracy: 0.7668 - loss: 0.6844 - val_accuracy: 0.8638 - val_loss: 0.4167
625/625 -----
Epoch 2/5 ----- 71s 113ms/step - accuracy: 0.8025 - loss: 0.5867 - val_accuracy: 0.8764 - val_loss: 0.3794
625/625 -----
Epoch 3/5 ----- 71s 113ms/step - accuracy: 0.8205 - loss: 0.5353 - val_accuracy: 0.8846 - val_loss: 0.3602
625/625 -----
Epoch 4/5 ----- 71s 113ms/step - accuracy: 0.8345 - loss: 0.4969 - val_accuracy: 0.8904 - val_loss: 0.3294
625/625 -----
Epoch 5/5 ----- 71s 113ms/step - accuracy: 0.8426 - loss: 0.4617 - val_accuracy: 0.8922 - val_loss: 0.3299
625/625 -----

Evaluasi Model: CNN Scratch
313/313 ----- 1s 3ms/step

Classification Report:
precision    recall  f1-score   support

airplane    0.80    0.65    0.72    1000
automobile  0.80    0.84    0.82    1000
bird        0.80    0.35    0.49    1000
cat         0.58    0.35    0.43    1000
deer        0.72    0.51    0.60    1000
dog         0.49    0.75    0.59    1000
frog        0.67    0.79    0.73    1000
horse       0.61    0.82    0.70    1000
ship        0.81    0.85    0.83    1000
truck       0.68    0.89    0.77    1000

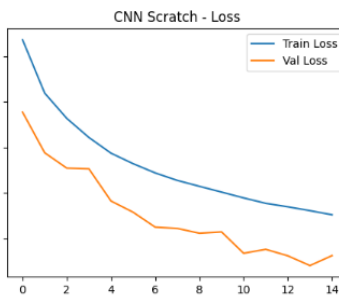
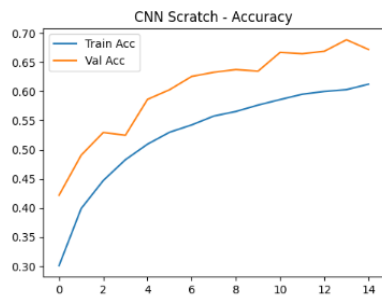
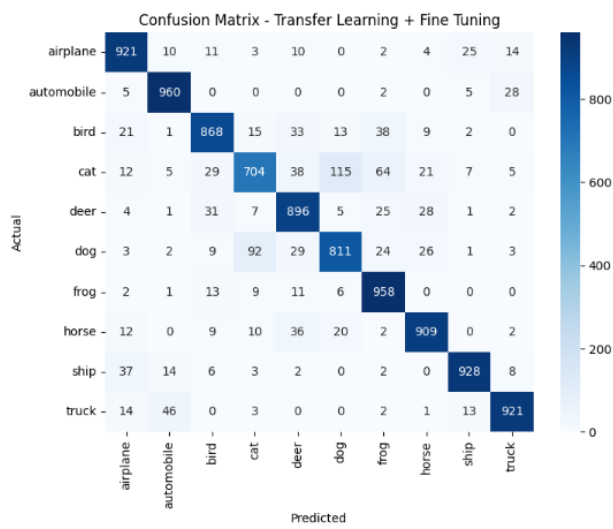
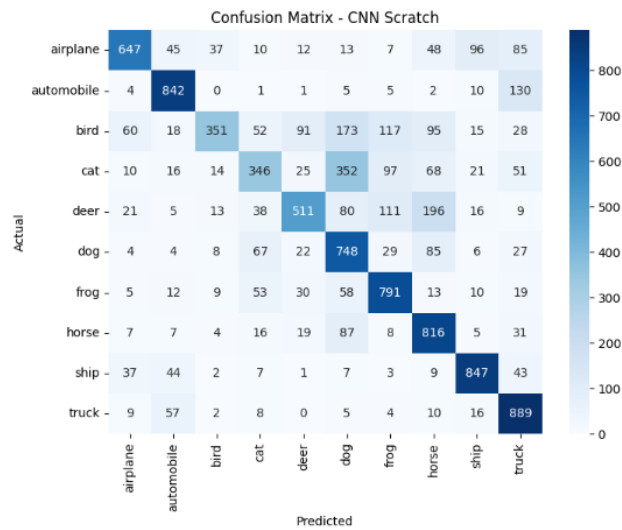
accuracy    0.68    10000
macro avg   0.70    0.68    0.67    10000
weighted avg 0.70    0.68    0.67    10000

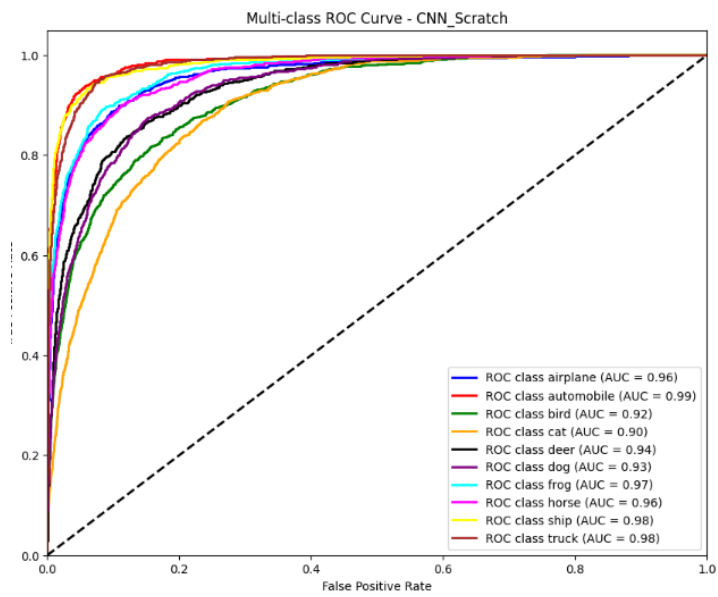
Evaluasi Model: Transfer Learning + Fine Tuning
313/313 ----- 14s 43ms/step

Classification Report:
precision    recall  f1-score   support

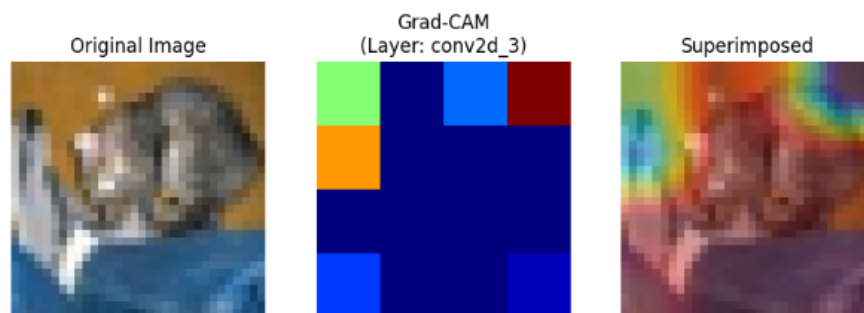
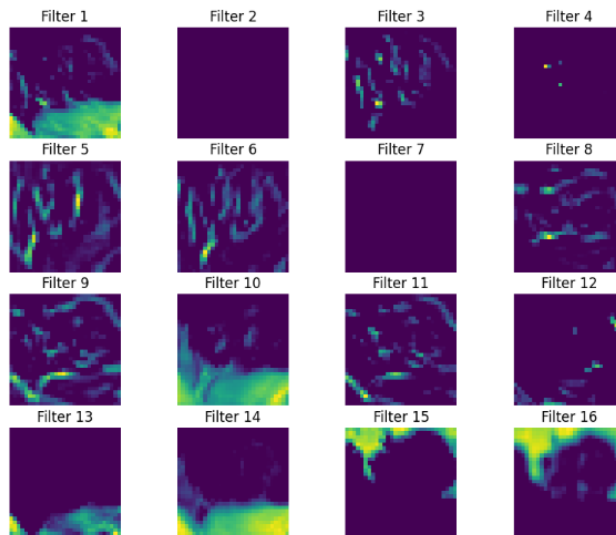
airplane    0.89    0.92    0.91    1000
automobile  0.92    0.96    0.94    1000
bird        0.89    0.87    0.88    1000
cat         0.83    0.70    0.76    1000
deer        0.85    0.90    0.87    1000
dog         0.84    0.81    0.82    1000
frog        0.86    0.96    0.90    1000
horse       0.91    0.91    0.91    1000
ship        0.95    0.93    0.94    1000
truck       0.94    0.92    0.93    1000

accuracy    0.89    10000
macro avg   0.89    0.89    0.89    10000
weighted avg 0.89    0.89    0.89    10000
```

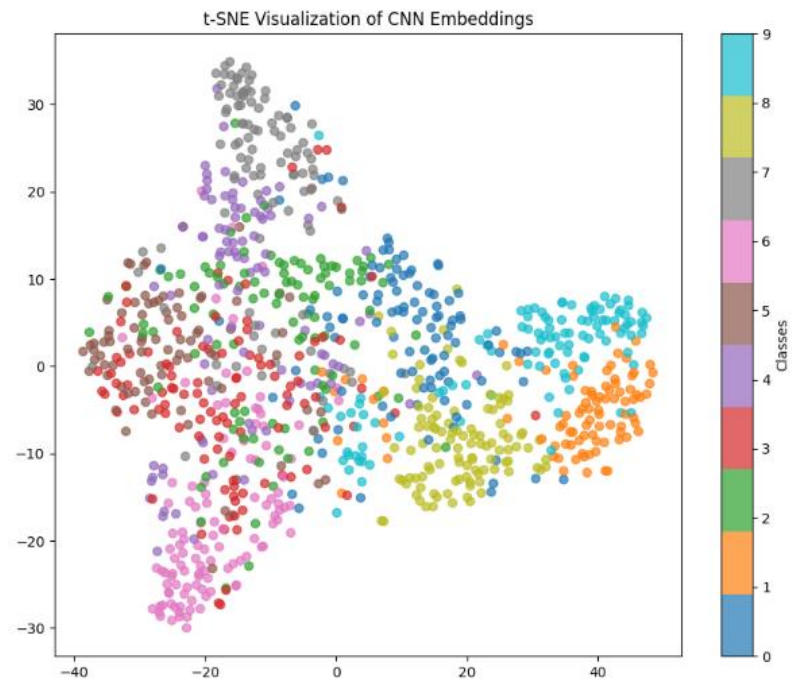




Feature Maps dari conv2d_1



Membuat visualisasi t-SNE dari fitur ekstrak...
32/32 16 10mc/cen



```
=====
TABEL PERBANDINGAN PERFORMA EKSPERIMEN (EPOCH TERAKHIR)
=====
```

Model	Train Acc	Val Acc	Train Loss	Val Loss	Waktu (Menit)
CNN Scratch (Adam)	0.6124	0.6719	1.1043	0.9251	7.84
Transfer Learning VGG16	0.8426	0.8922	0.4617	0.3299	16.75

```
=====
```